算法设计与分析

高翔

gaoxg@nwpu.edu.cn

第1章 概述

目 录

1.1 算法的概念

1.2 算法分析

1.3 算法设计工具—STL

1.1 算法的概念

1.1.1 什么是算法

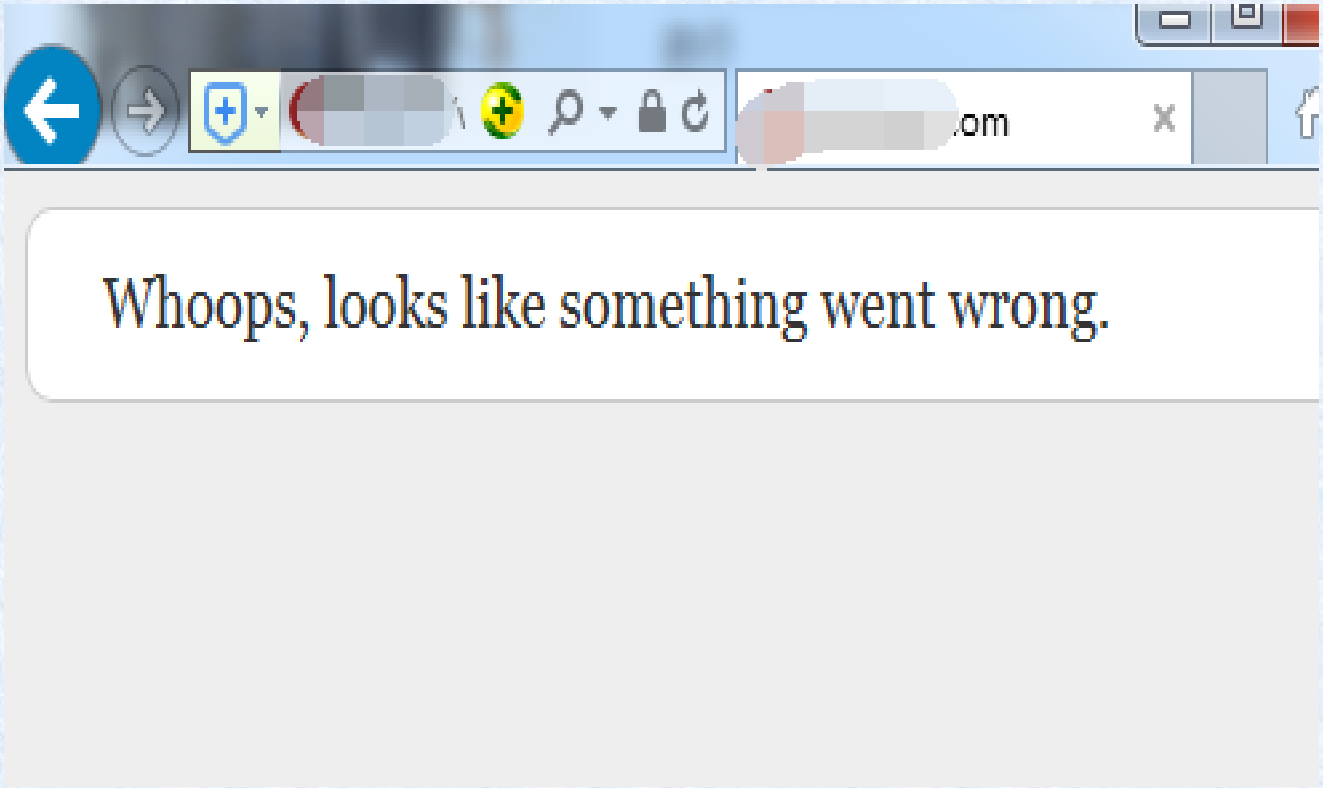
算法是求解问题的一系列计算步骤，用来将输入数据转换成输出结果：

如果一个算法对其每一个输入实例，都能输出正确的结果并停止，则称它是正确的。

算法设计应满足以下几条目标:

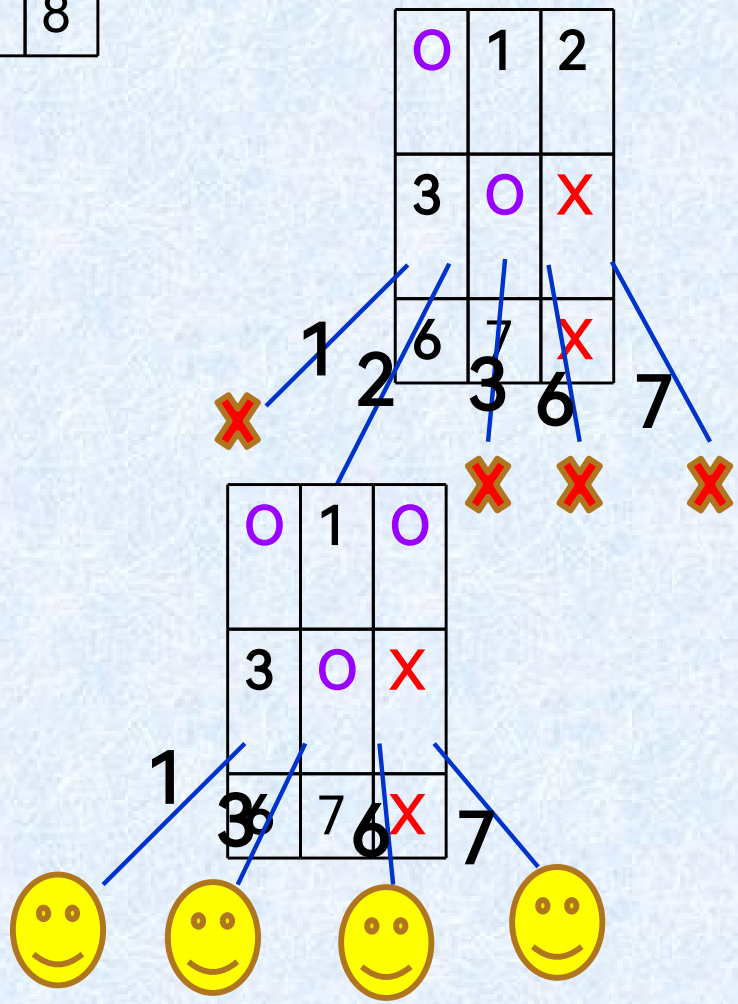
- 正确性
- 可使用性
- 可读性
- 健壮性
- 高效率与低存储量需求

正确性

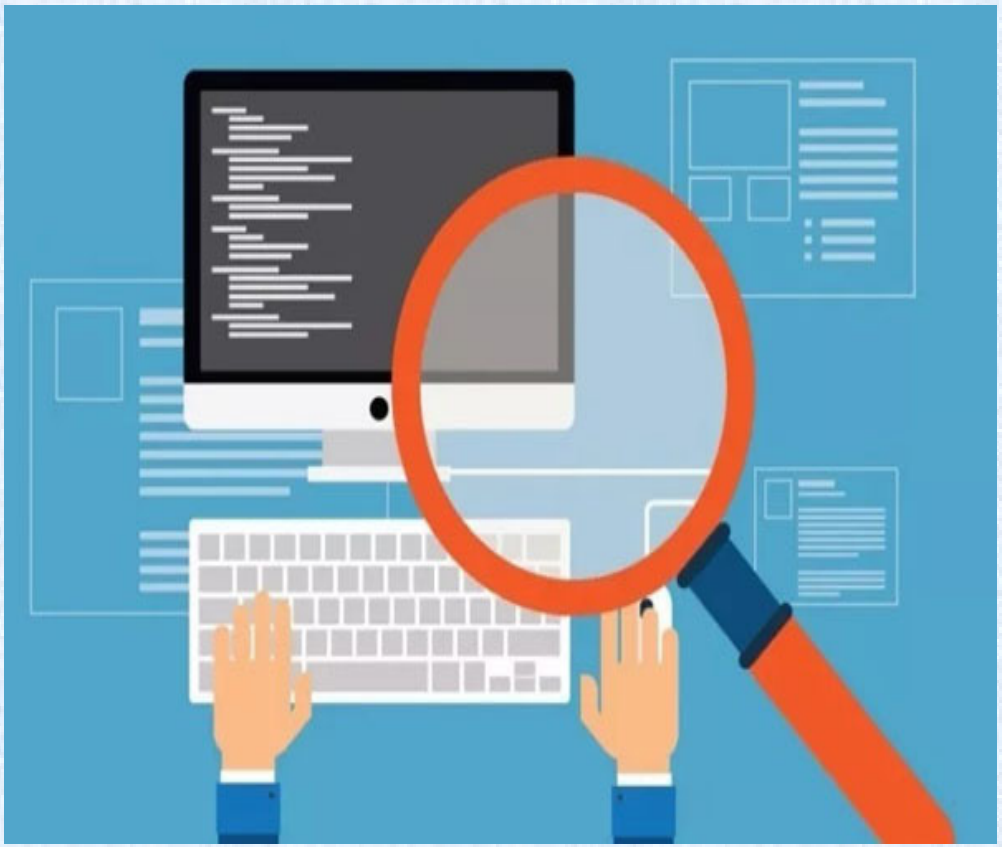


可使用性

0	1	2
3	4	5
6	7	8



可读性



健壮性

特斯拉为何总是钟爱白色的卡车?

健壮性

SEARCH

AMAZIN' BUT TRUE
A NEW YORK METS BASEBALL PODCAST

NEW YORK POST

Prince William asked to not walk beside Harry at Philip's funeral

Russia reportedly adds to military muscle in Black Sea region amid US...

'Consensual incest' should be decriminalized, advocates say

NEWS

Two killed in huge blaze after driverless Tesla crashes into tree

By Jorge Fitz-Gibbon

April 18, 2021 | 2:51pm | Updated

Elon Musk @elonmusk · 9小时

Your research as a private individual is better than professionals @WSJ!

Data logs recovered so far show Autopilot was not enabled & this car did not purchase FSD.

Moreover, standard Autopilot would require lane lines to turn on, which this street did not have.

1,059 3,451 1.3万

健壮性

16:28 ··· 85%

insideevs.com/news/630445/tesla-fsd-crash-san-francisco-footage

Home > Tesla > Model S > News

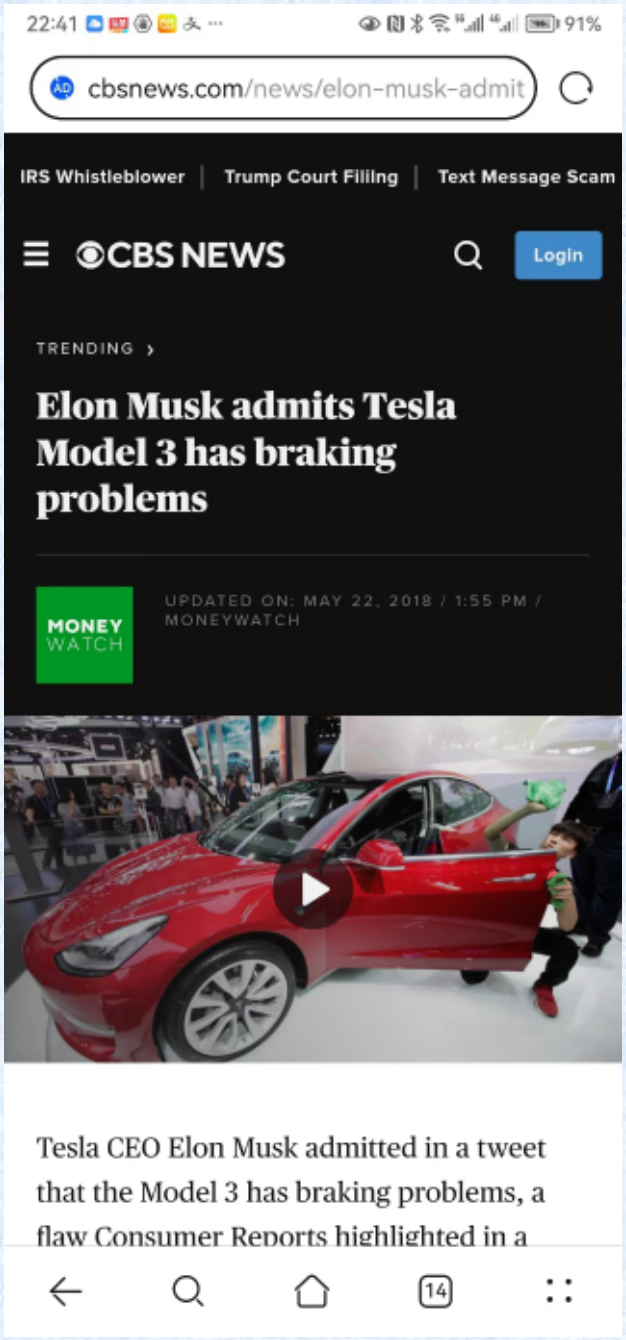
Here's What Happened In The 8-Car Pileup In SF Blamed On Tesla's FSD

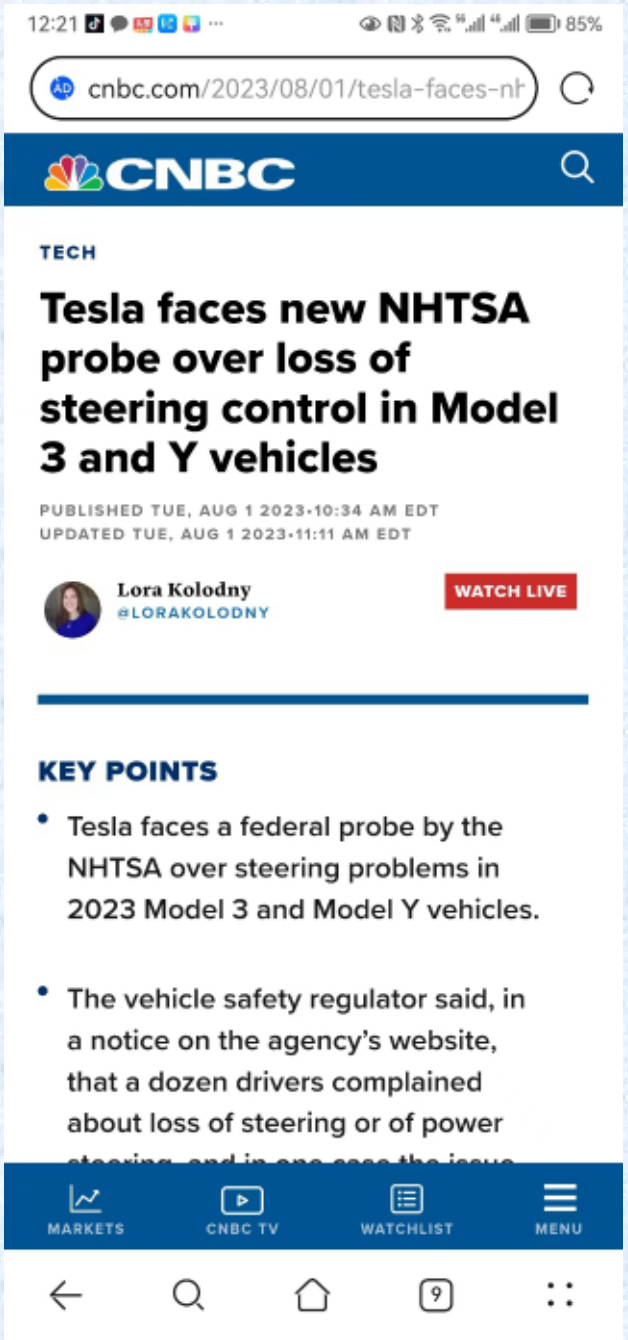
Surveillance footage shows the Tesla Model S switching lanes and putting on its brakes.

Jan 10, 2023 at 11:45pm ET

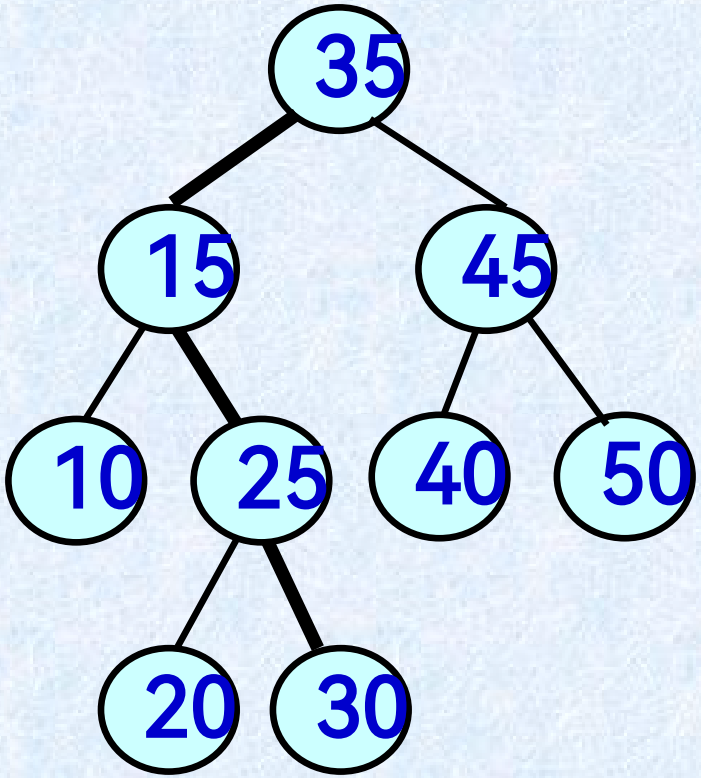
By: Iulian Dnistran

健壮性


22:41
91%
cbsnews.com/news/elon-musk-admit
IRS Whistleblower | Trump Court Filing | Text Message Scam
CBS NEWS
Login
TRENDING ›
Elon Musk admits Tesla Model 3 has braking problems
MONEY WATCH
UPDATED ON: MAY 22, 2018 / 1:55 PM / MONEYWATCH
Tesla CEO Elon Musk admitted in a tweet that the Model 3 has braking problems, a flaw Consumer Reports highlighted in a


12:21
85%
cnbc.com/2023/08/01/tesla-faces-nh
CNBC
TECH
Tesla faces new NHTSA probe over loss of steering control in Model 3 and Y vehicles
PUBLISHED TUE, AUG 1 2023·10:34 AM EDT
UPDATED TUE, AUG 1 2023·11:11 AM EDT
Lora Kolodny
@LORAKOLODNY
WATCH LIVE
KEY POINTS
Tesla faces a federal probe by the NHTSA over steering problems in 2023 Model 3 and Model Y vehicles.
The vehicle safety regulator said, in a notice on the agency's website, that a dozen drivers complained about loss of steering or of power
MARKETS
CNBC TV
WATCHLIST
MENU

高效率与低存储量需求



35	45	15	10	25	40	50	20	30

【例1.1】以下算法用于在带头结点的单链表*h*中查找第一个值为*x*的结点，找到后返回其逻辑序号（从1计起），否则返回0。分析该算法存在的问题。

```
int findx(LNode *h;int x)
{   LNode *p=h->next;
      int i=0;
      while (p->data!=x)
      {     i++;
            p=p->next;
      }
      return i;
}
```

- 当单链表中首结点值为x时，该算法返回0，此时应该返回逻辑序号1。
- 当单链表中不存在值为x的结点时，该算法执行出错，因为p为NULL时仍执行p=p->next。

所以该算法不满足正确性和健壮性。应改为：

```
int findx(LNode *h;int x)
{   LNode *p=h->next;
      int i=0;
      while (p->data!=x)
      {     i++;
            p=p->next;
      }
      return i;
}
```

```
int findx(LNode *h;int x)
{   LNode *p=h->next;
      int i=1;
      while (p!=NULL && p->data!=x)
      {     i++;
            p=p->next;
      }
      if (p==NULL)
            return 0;
      else
            return i;
}
```

算法具有以下5个重要特征:

- 有限性
- 确定性
- 可行性
- 输入性
- 输出性

【例1.2】有下列两段描述：

描述1：

```
void exam1()
{  int n;
    n=2;
    while (n%2==0)
        n=n+2;
    printf("%d\n",n);
}
```

描述2：

```
void exam2()
{  int x,y;
    y=0;
    x=5/y;
    printf("%d,%d\n",x,y);
}
```

这两段描述均不能满足算法的特征，试问它们违反了算法的哪些特征？

解：（1）是一个死循环，违反了算法的有限性特征。

（2）出现除零错误，违反了算法的可行性特征。

```
void exam1()
{  int n;
    n=2;
    while (n%2==0)
        n=n+2;
    printf("%d\n",n);
}
```

```
void exam2()
{  int x,y;
    y=0;
    x=5/y;
    printf("%d,%d\n",x,y);
}
```

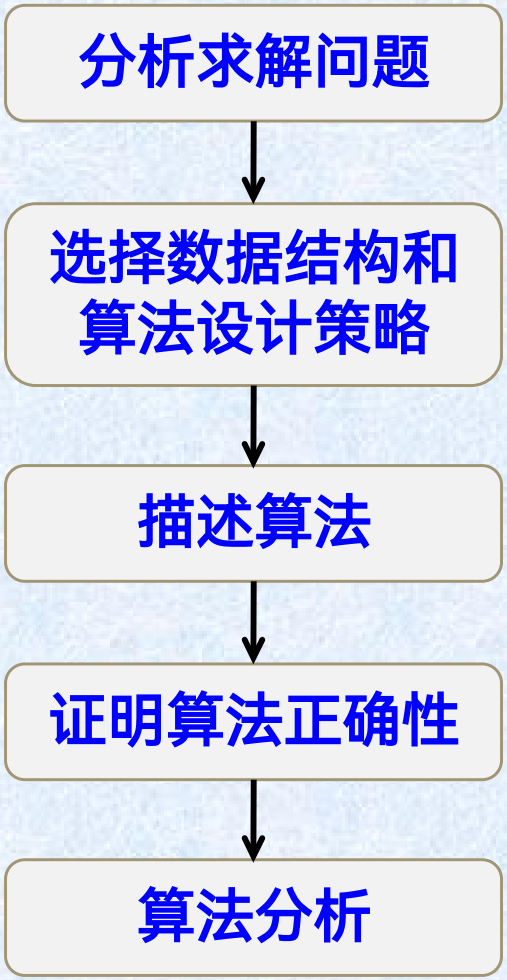
1.1.2 算法和数据结构

算法与数据结构既有联系又有区别。

联系：数据结构是算法设计的基础。算法的操作对象是数据结构，在设计算法时，通常要构建适合这种算法的数据结构。数据结构设计主要是选择数据的存储方式，如确定求解问题中的数据采用数组存储还是采用链表存储等。算法设计就是在选定的存储结构上设计一个满足要求的好算法。

区别：数据结构关注的是数据的逻辑结构、存储结构以及基本操作，而算法更多的是关注如何在数据结构的基础上解决实际问题。算法是编程思想，数据结构则是这些思想的逻辑基础。

1.1.3 算法设计的基本步骤



1.2 算法分析

算法分析是分析算法占用计算机资源的情况。

所以算法分析的两个主要方面是分析算法的时间复杂度和空间复杂度。

1.2.1 算法时间复杂度分析

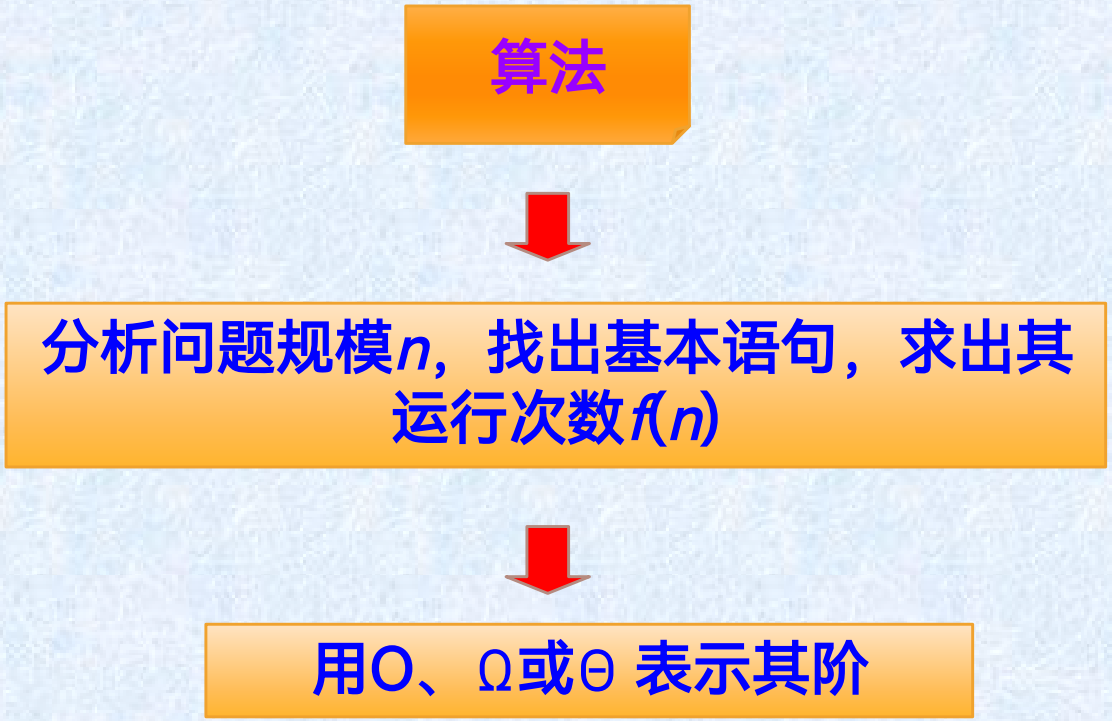
1. 时间复杂度分析概述

一个算法是由控制结构（顺序、分支和循环3种）和原操作（指固有数据类型的操作）构成的，算法的运行时间取决于两者的综合效果。

```
bool Solve(double a[][MAX],int m,int n,double &s)
{
   int i; s=0;                          ← 顺序结构
   if (m!=n) return false;              ← 分支结构
   for (i=0;i<m;i++)                    ← 循环结构
    s+=a[i][i];
   return true;                         ← 顺序结构
}
```

设n为算法中的问题规模，通常用大O、大Ω和Θ等三种渐进符号表示算法的执行时间与n之间的一种增长关系。

分析算法时间复杂度的一般步骤：



2. 渐进符号（O、Ω和Θ）

定义1（大O符号）， $f(n)=O(g(n))$（读作“$f(n)$是$g(n)$的大O”）当且仅当存在正常量c和n_0，使当$n \geq n_0$时，$f(n) \leq cg(n)$，即$g(n)$为$f(n)$的上界。

如$3n+2=O(n)$，因为当$n \geq 2$时，$3n+2 \leq 4n$。

$10n^2+4n+2=O(n^4)$，因为当$n \geq 2$时，$10n^2+4n+2 \leq 10n^4$。

大O符号用来描述增长率的上界，表示$f(n)$的增长最多像$g(n)$ 增长的那样快，也就是说，当输入规模为n时，算法消耗时间的最大值。这个上界的阶越低，结果就越有价值，所以，对于$10n^2+4n+2$，$O(n^2)$比$O(n^4)$ 有价值。

一个算法的时间用大O符号表示时，总是采用最有价值的$g(n)$表示，称之为“紧凑上界”或“紧确上界”。

一般地，如果$f(n)=a_m n^m+a_{m-1}n^{m-1}+\ldots+a_1 n+a_0$，有$f(n)=O(n^m)$。

Asymptotic Analysis: Big-oh

Definition:

For T(n) a non-negatively valued function, T(n) is in the set O($f(n)$) if there exist two positive constants c and n_0 such that T(n) <= $cf(n)$ for all $n > n_0$.

Usage: The algorithm is in O(n^2) in [best, average, worst] case.

Meaning: For all data sets big enough (i.e., $n>n_0$), the algorithm always executes in less than $cf(n)$ steps in [best, average, worst] case.

Big-oh Notation

Big-oh notation indicates an upper bound.

Example: If $\mathbf{T}(n) = 3n^2$ then $\mathbf{T}(n)$ is in $O(n^2)$.

Wish tightest upper bound:

For example, while $\mathbf{T}(n) = 3n^2$ is in $O(n^3)$,
we prefer $O(n^2)$.

Big-Oh Examples

Example 1: Finding value X in an array (average cost).

$T(n) = c_s n/2$.

For all values of $n > 1$, $c_s n/2 <= c_s n$.

Therefore, by the definition, $T(n)$ is in $O(n)$ for $n_0 = 1$ and $c = c_s$.

Big-Oh Examples

Example 2: $T(n) = c_1 n^2 + c_2 n$ in average case.

$c_1 n^2 + c_2 n <= c_1 n^2 + c_2 n^2 <= (c_1 + c_2) n^2$ for all $n > 1$.

$T(n) <= cn^2$ for $c = c_1 + c_2$ and $n_0 = 1$.

Therefore, $T(n)$ is in $O(n^2)$ by the definition.

Example 3: $T(n) = c$. We say this is in $O(1)$.

A Common Misunderstanding

"The best case for my algorithm is n=1 because that is the fastest." WRONG!

Big-oh refers to a growth rate as n grows to ∞.

Best case is defined as which input of size n is cheapest among all inputs of size n.

定义2（大Ω符号）， $f(n)=\Omega(g(n))$（读作“$f(n)$是$g(n)$的大Ω”）当且仅当存在正常量c和n_0，使当$n\geq n_0$时，$f(n)\geq cg(n)$，即$g(n)$为$f(n)$的下界。

如$3n+2=\Omega(n)$，因为当$n\geq 1$时，$3n+2\geq 3n$。

$10n^2+4n+2=\Omega(n^2)$，因为当$n\geq 1$时，$10n^2+4n+2\geq n^2$。

大Ω符号用来描述增长率的下界，表示$f(n)$的增长最少像$g(n)$ 增长的那样快，也就是说，当输入规模为n时，算法消耗时间的最小值。

与大O符号对称，这个下界的阶越高，结果就越有价值，所以，对于$10n^2+4n+2$，$\Omega(n^2)$比$\Omega(n)$ 有价值。一个算法的时间用大Ω符号表示时，总是采用最有价值的$g(n)$表示，称之为“紧凑下界”或“紧确下界”。

一般地，如果$f(n)=a_m n^m+a_{m-1}n^{m-1}+\ldots+a_1 n+a_0$，有$f(n)=\Omega(n^m)$。

Big-Omega

Definition: For **T**(*n*) a non-negatively valued function, **T**(*n*) is in the set Ω(*g*(*n*)) if there exist two positive constants *c* and n_0 such that **T**(*n*) >= *cg*(*n*) for all $n > n_0$.

Meaning: For all data sets big enough (i.e.,$n > n_0$), the algorithm always executes in more than *cg*(*n*) steps.

Big-Omega notation indicates an lower bound.

Big-Omega Example

$T(n) = c_1 n^2 + c_2 n.$

$c_1 n^2 + c_2 n >= c_1 n^2$ for all $n > 1$.
$T(n) >= cn^2$ for $c = c_1$ and $n_0 = 1$.

Therefore, $T(n)$ is in $\Omega(n^2)$ by the definition.

We want the greatest lower bound.

定义3（大Θ符号）， $f(n)=\Theta(g(n))$（读作“$f(n)$是$g(n)$的大Θ”）当且仅当存在正常量c_1、c_2和n_0，使当$n\geq n_0$时，有$c_1g(n)\leq f(n)\leq c_2g(n)$，即$g(n)$与$f(n)$的**同阶**。

如$3n+2=\Theta(n)$，$10n^2+4n+2=\Theta(n^2)$。

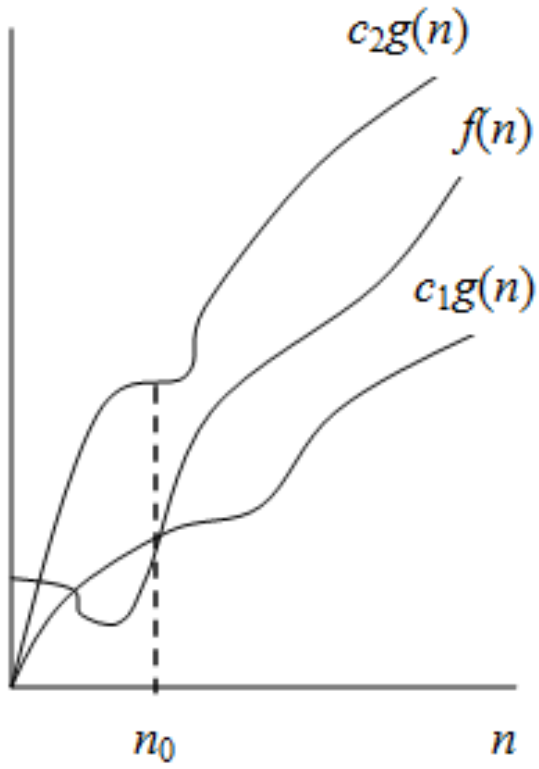
一般地，如果$f(n)=a_mn^m+a_{m-1}n^{m-1}+\ldots+a_1n+a_0$，有$f(n)=\Theta(n^m)$。

大Θ符号比大O符号和大Ω符号都精确，$f(n)=\Theta(g(n))$，当且仅当$g(n)$既是$f(n)$的上界又是$f(n)$的下界。

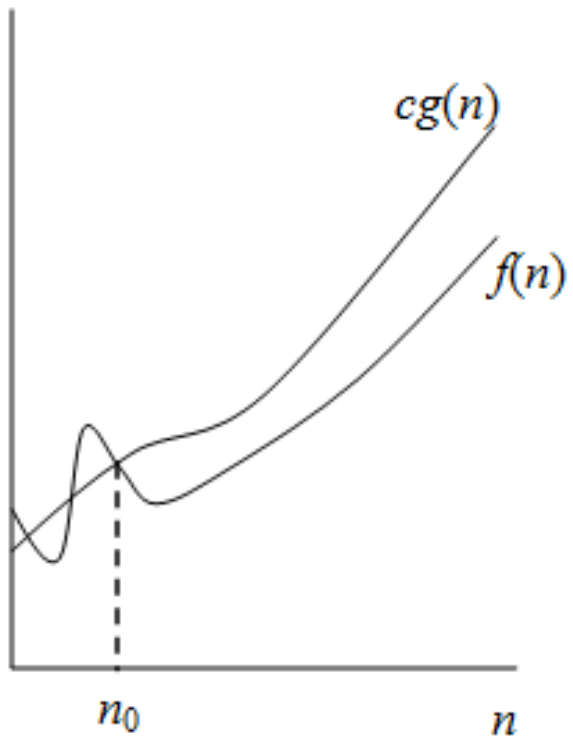
Theta Notation

When big-Oh and Ω meet, we indicate this by using Θ (big-Theta) notation.

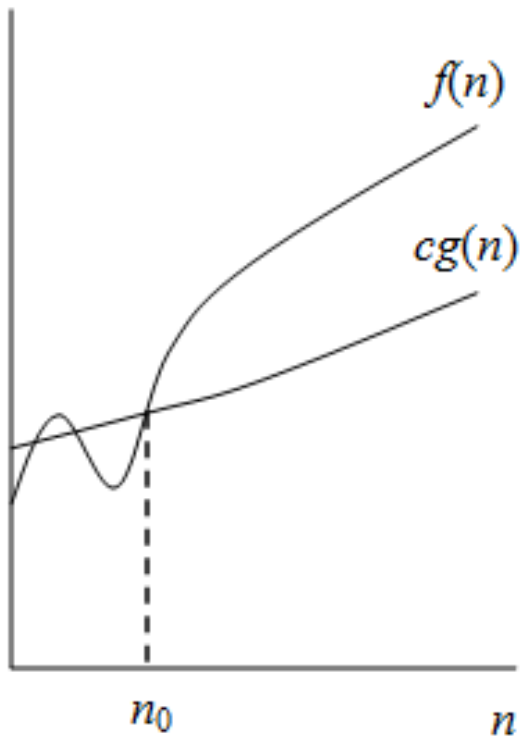
Definition: An algorithm is said to be Θ($h(n)$) if it is in O($h(n)$) and it is in Ω($h(n)$).



(a) $f(n)=\Theta(g(n))$



(b) $f(n)=O(g(n))$



(c) $f(n)=\Omega(g(n))$

A Common Misunderstanding

Confusing worst case with upper bound.

Upper bound refers to a growth rate.

Worst case refers to the worst input from among the choices for possible inputs of a given size.

Simplifying Rules

1. If $f(n)$ is in $O(g(n))$ and $g(n)$ is in $O(h(n))$, then $\mathbf{f(n)}$ is in $\mathbf{O(h(n))}$.
2. If $f(n)$ is in $O(kg(n))$ for any constant $k > 0$, then $\mathbf{f(n)}$ is in $\mathbf{O(g(n))}$.
3. If $f_1(n)$ is in $O(g_1(n))$ and $f_2(n)$ is in $O(g_2(n))$, then $\mathbf{(f_1 + f_2)(n)}$ is in $\mathbf{O(\max(g_1(n), g_2(n)))}$.
4. If $f_1(n)$ is in $O(g_1(n))$ and $f_2(n)$ is in $O(g_2(n))$ then $\mathbf{f_1(n)f_2(n)}$ is in $\mathbf{O(g_1(n)g_2(n))}$.

Running Time Examples

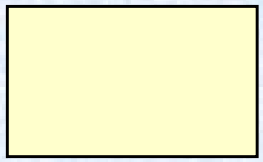
Example 1:

```
a = b;
```

This assignment takes constant time, so it is $\Theta(1)$.

Example 2:

```
sum = 0;
for (i=1; i<=n; i++)
  sum += n;
```

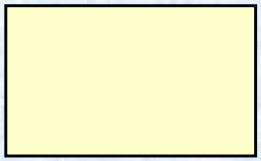


Running Time Examples

Example 3:

```
sum = 0;
for (j=1; j<=n; j++)
  for (i=1; i<=j; i++)
    sum++;

for (k=0; k<n; k++)
  A[k] = k;
```

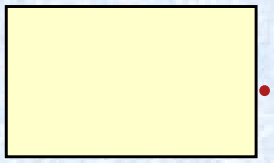


Running Time Examples

Example 4:

```
sum1 = 0;
for (i=1; i<=n; i++)
  for (j=1; j<=n; j++)
    sum1++;

sum2 = 0;
for (i=1; i<=n; i++)
  for (j=1; j<=i; j++)
    sum2++;
```

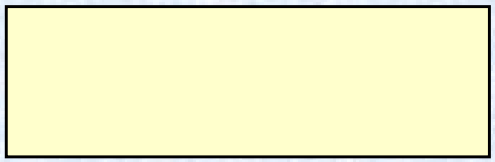


Running Time Examples

Example 5:

```
sum1 = 0;
for (k=1; k<=n; k*=2)
  for (j=1; j<=n; j++)
    sum1++;

sum2 = 0;
for (k=1; k<=n; k*=2)
  for (j=1; j<=k; j++)
    sum2++;
```



Other Control Statements

while loop: Analyze like a **for** loop.

if statement: Take greater complexity of **then/else** clauses.

switch statement: Take complexity of most expensive case.

Subroutine call: Complexity of the subroutine.

【例1.3】分析以下算法的时间复杂度：

```
void fun(int n)
{  int s=0,i,j,k;
   for (i=0;i<=n;i++)
            for (j=0;j<=i;j++)
                for (k=0;k<j;k++)
                        s++;
}
```

解：该算法的基本语句是s++，所以有：

$$f(n)=\sum_{i=0}^{n}\sum_{j=0}^{i}\sum_{k=0}^{j-1}1=\sum_{i=0}^{n}\sum_{j=0}^{i}(j-1-0+1)=\sum_{i=0}^{n}\sum_{j=0}^{i}j$$

$$=\sum_{i=0}^{n}\frac{i(i+1)}{2}=\frac{1}{2}(\sum_{i=0}^{n}i^2+\sum_{i=0}^{n}i)=\frac{2n^3+6n^2+4n}{12}=O(n^3)$$

则该算法的时间复杂度为$O(n^3)$。

$(n+1)^3=n^3+3n^2+3n+1$

$(n+1)^3-n^3=3n^2+3n+1$
$n^3-(n-1)^3=3(n-1)^2+3(n-1)+1$
$(n-1)^3-(n-2)^3=3(n-2)^2+3(n-2)+1$
...
$3^3-2^3=3*2^2+3*2+1$
$2^3-1^3=3*1^2+3*1+1$
两边分别相加得
$(n+1)^3-1^3$
$=3*(1^2+2^2+...+n^2)+3(1+2+...+n)+1*n$
$(n^3+3n^2+3n)-n=3Sn$
$Sn=（n^3+3n^2+2n）/3$

$s_n = \sum_{i=0}^{n} i^2+\sum_{i=0}^{n} i$

3. 算法的最好、最坏和平均情况

定义4 设一个算法的输入规模为n，D_n是所有输入的集合，任一输入$I \in D_n$，$P(I)$是I出现的概率，有 $\sum_{I \in D_n} P(I) = 1$，$T(I)$是算法在输入$I$下所执行的基本语句次数，则该算法的平均执行时间为：

$A(n) = \sum_{I \in D_n} P(I)*T(I)$ 。

也就是说算法的平均情况是指用各种特定输入下的基本语句执行次数的带权平均值。

算法的最好情况为：$G(n)=\min_{I\in D_n}\{T(I)\}$，是指算法在所有输入下所执行基本语句的最少次数。

算法的最坏情况为：$W(n)=\max_{I\in D_n}\{T(I)\}$，是指算法在所有输入下所执行基本语句的最大次数。

【例1.4】采用顺序查找方法，在长度为n的一维实型数组$a[0..n-1]$中查找值为x的元素。即从数组的第一个元素开始，逐个与被查值x进行比较。找到后返回1，否则返回0，对应的算法如下：

```
int Find(double a[],int n,double x)
{  int i=0;
   while (i<n)
   {  if (a[i]==x) break;
      i++;
   }
   if (i<n) return 1;
   else return 0;
}
```

回答以下问题：

（1）分析该算法在等概率情况下成功查找到值为x的元素的最好、最坏和平均时间复杂度。

（2）假设被查值x在数组a中的概率是q，求算法的时间复杂度。

```
int Find(double a[],int n,double x)
{  int i=0;
   while (i<n)
   {  if (a[i]==x) break;
      i++;
   }
   if (i<n) return 1;
   else return 0;
}
```

解： （1）算法的while循环中的if语句是基本语句。a数组中有n个元素，当第一个元素$a[0]$等于x，此时基本语句仅执行一次，此时呈现最好的情况，即$G(n)=O(1)$。

当a中最后一个元素$a[n-1]$等于x，此时基本语句执行n次，此时呈现最坏的情况，即$W(n)=O(n)$。

```
int Find(double a[],int n,double x)
{  int i=0;
   while (i<n)
   {  if (a[i]==x) break;
      i++;
   }
   if (i<n) return 1;
   else return 0;
}
```

对于其他情况，假设查找每个元素的概率相同，则$P(a[i])=1/n$（$0\le i\le n-1$），而成功找到$a[i]$元素时基本语句正好执行$i+1$次，所以：

$$A(n)=\sum_{i=0}^{n-1}\frac{1}{n}(i+1)=\frac{1}{n}\sum_{i=0}^{n-1}(i+1)=\frac{n+1}{2}=O(n)。$$

（2）当被查值x在数组a中的概率为q时，算法执行有$n+1$种情况，即n种成功查找和一种不成功查找。

对于成功查找，假设是等概率情况，则元素$a[i]$被查找到的概率$P(a[i])=q/n$，成功找到$a[i]$元素时基本语句正好执行$i+1$次。

对于不成功查找，其概率为$1-q$，不成功查找时基本语句正好执行n次。

所以：

$$A(n)=\sum_{I\in D_n} P(I)*T(I)$$

$$=\sum_{i=0}^{n} P(I)*T(I)=\sum_{i=0}^{n-1}\frac{q}{n}(i+1)+(1-q)n=\frac{(n+1)q}{2}+(1-q)n$$

如果已知需要查找的x有一半的机会在数组中，此时$q=1/2$，则

$A(n)=[(n+1)/4]+n/2\approx 3n/4$。

4. 非递归算法的时间复杂度分析

对于非递归算法，分析其时间复杂度相对比较简单，关键是求出代表算法执行时间的表达式。

通常是算法中基本语句的执行次数，是一个关于问题规模*n*的表达式，然后用渐进符号来表示这个表达式即得到算法的时间复杂度。

【例1.6】给出以下算法的时间复杂度。

```
void func(int n)
{   int i=1,k=100;
    while (i<=n)
    {  k++;
       i+=2;
    }
}
```

解：算法中基本语句是while循环内的语句。设while循环语句执行的次数为m，i从1开始递增，最后取值为$1+2m$，有：

$i=1+2m\leq n$

$f(n)=m\leq(n-1)/2=O(n)$。

该算法的时间复杂度为$O(n)$。

5. 递归算法的时间复杂度分析

递归算法是采用一种分而治之的方法，把一个“大问题”分解为若干个相似的“小问题”来求解。

对递归算法时间复杂度的分析，关键是根据递归过程建立递推关系式，然后求解这个递推关系式，得到一个表示算法执行时间的表达式，最后用渐进符号来表示这个表达式即得到算法的时间复杂度。

【例1.7】有以下递归算法：

```
void mergesort(int a[],int i,int j)
{   int m;
    if (i!=j)
    {         m=(i+j)/2;
        mergesort(a,i,m);
        mergesort(a,m+1,j);
        merge(a,i,j,m);
    }
}
```

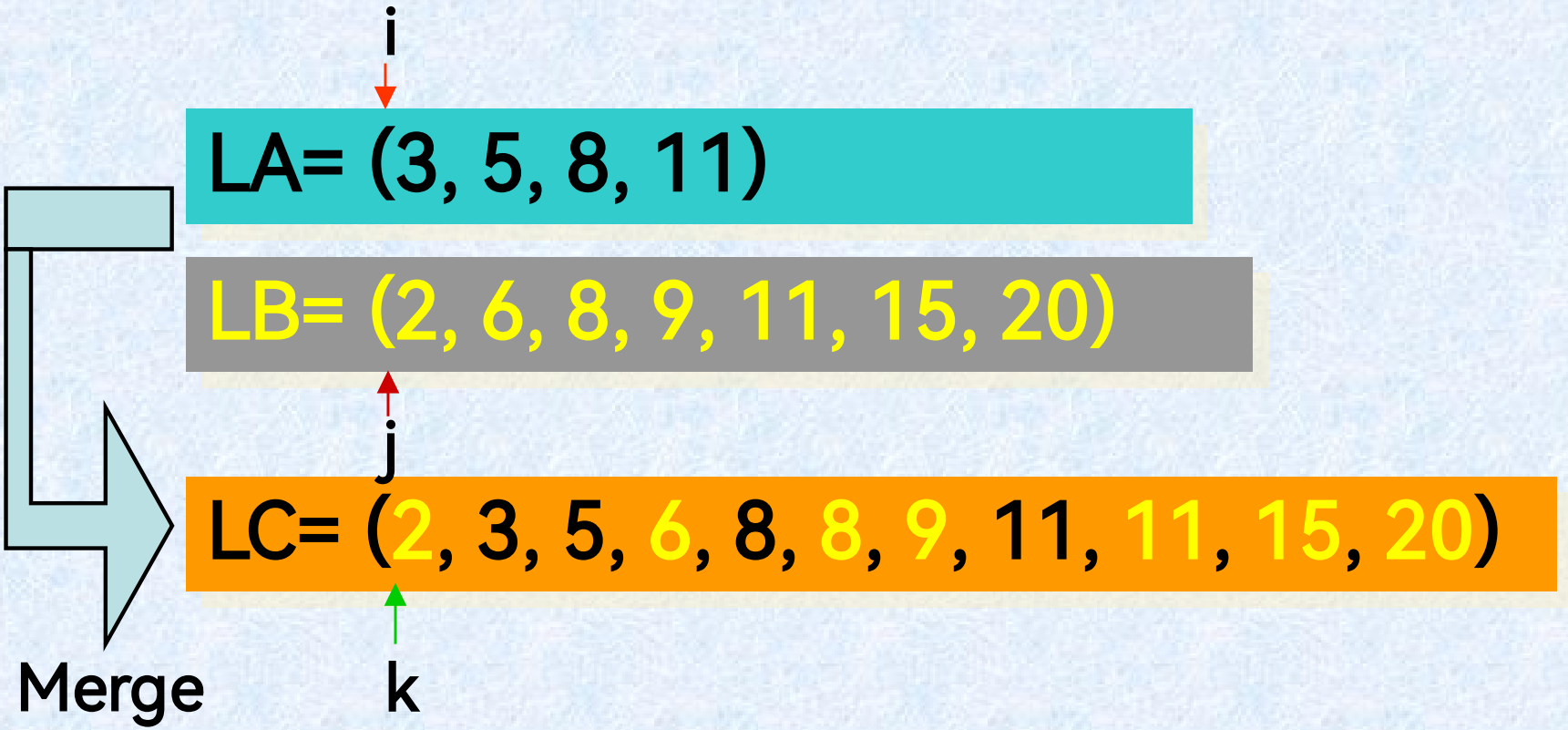
其中，mergesort()用于数组$a[0..n-1]$（设$n=2^k$，这里的k为正整数）的归并排序，调用该算法的方式为：

mergesort(a，0，n-1);

另外merge(a，i，j，m)用于两个有序子序列$a[i..j]$和$a[j+1..m]$的有序合并，是非递归函数，它的时间复杂度为O(n)（这里$n=j-i+1$）。分析上述调用的时间复杂度。

Sorted lists merging

Merge two sorted lists into a new list and the new one is also sorted as before.



How about time complexity?

Implementation: Sorted lists merging

```
void MergeSqList ( SeqList *la, SeqList *lb, SeqList *lc)
{
    int i=0, j=0, k=0;
    m = la->length; n = lb->length; lc->length=0;
    while ( i < m && j<n) {
        x = retrieve_seq (la, i);       // Get an item x from la
        y = retrieve_seq (lb, j);       // Get an item y from
    lb
        if (x<=y) {
            Insert_Seq(lc, x, k); i++; k++;    // Insert x into
    lc
        }
        else {
            Insert_Seq(lc, y, k); j++; k++;    // Insert y into
    lc
        }
    }
    while (i<m) {x = retrieve_seq (la, i); Insert_Seq(lc, x, k
    ); i++; k++;}
    while (j<n)  {y = retrieve_seq (lb, j); Insert_Seq(lc, y, k
    ); j++; k++;}
}
```

解： 设调用mergesort(a，0，n-1)的执行时间为$T(n)$，由其执行过程得到以下求执行时间的递归关系（递推关系式）：

$T(n)=O(1)$ 当$n=1$

$T(n)=2T(n/2)+O(n)$ 当$n>1$

```
void mergesort(int a[],int i,int j)
{  int m;
   if (i!=j)
   {  m=(i+j)/2;
      mergesort(a,i,m);
      mergesort(a,m+1,j);
      merge(a,i,j,m);
   }
}
```

其中，$O(n)$为merge()所需的时间，设为cn（c为正常量）。因此：

$$
\begin{aligned}
T(n) &= 2T(n/2)+cn=2[2T(n/2^2)+cn/2]+cn=2^2T(n/2^2)+2cn \\
&= 2^3T(n/2^3)+3cn \\
&= \cdots \\
&= 2^kT(n/2^k)+kcn \\
&= nO(1)+cn\log_2 n=n+cn\log_2 n \quad //\text{这里假设}n=2^k\text{，则}k=\log_2 n \\
&= O(n\log_2 n)
\end{aligned}
$$

1.2.2 算法空间复杂度分析

一个算法的存储量包括形参所占空间和临时变量所占空间。在对算法进行存储空间分析时，只考察临时变量所占空间。

例如，有以下算法，其中临时空间为变量*i*、maxi占用的空间。所以，空间复杂度是对一个算法在运行过程中临时占用的存储空间大小的量度，一般也作为问题规模*n*的函数，以数量级形式给出，记作：

$S(n)=O(g(n))$、$\Omega(g(n))$或$\Theta(g(n))$

其中渐进符号的含义与时间复杂度中的含义相同。

```
int max(int a[], int n)
{   int i, maxi=0;
    for (i=1;i<n;i++)
            if (a[i]>a[maxi])
                maxi=i;
    return a[maxi];
}
```

函数体内分配的变量空间为临时空间，不计形参占用的空间，这里的仅计*i*、maxi变量的空间，其空间复杂度为O(1)。

算法空间复杂度的分析方法与前面介绍的时间复杂度分析方法相似。

【例1.9】分析例1.6算法的空间复杂度。

```
void func(int n)
{   int i=1,k=100;
    while (i<=n)
    {           k++;
                i+=2;
    }
}
```

解： 该算法是一个非递归算法，其中只临时分配了*i*、*k*两个变量的空间，它与问题规模*n*无关，所以其空间复杂度均为O(1)，即该算法为原时工作算法。

【例1.10】有如下递归算法，分析调用

maxelem(*a*，0，*n*–1)

的空间复杂度。

```
int maxelem(int a[],int i,int j)
{   int mid=(i+j)/2,max1,max2;
    if (i<j)
    {       max1=maxelem(a,i,mid);
            max2=maxelem(a,mid+1,j);
            return (max1>max2)?max1:max2;
    }
    else return a[i];
}
```

解：执行该递归算法需要多次调用自身，每次调用只临时分配3个整型变量的空间（O(1)）。

设调用maxelem(*a*，0，*n*–1)的空间为*S*(*n*)，有：

```
int maxelem(int a[],int i,int j)
{   int mid=(i+j)/2,max1,max2;
    if (i<j)
    {           max1=maxelem(a,i,mid);
                max2=maxelem(a,mid+1,j);
                return (max1>max2)?max1:max2;
    }
    else return a[i];
}
```

$S(n)=O(1)$ 当$n=1$

$S(n)=2S(n/2)+O(1)$ 当$n>1$

则：$S(n)=2S(n/2)+1=2[2S(n/2^2)+1]+1=2^2S(n/2^2)+1+2^1$

$=2^3S(n/2^3)+1+2^1+2^2$

$=\ldots$

$=2^kS(n/2^k)+1+2^1+2^2+\ldots+2^{k-1}$（设$n=2^k$，即$k=\log_2 n$）

$=2^k+2^k-1=2n-1=O(n)$

1.3 算法设计工具—STL

1.3.1 STL概述

STL主要由container（容器）、algorithm（算法）和iterator（迭代器）三大部分构成，容器用于存放数据对象（元素），算法用于操作容器中的数据对象。

1. 什么是STL容器

一个STL容器就是一种数据结构，如链表、栈和队列等，这些数据结构在STL中都已经实现好了，在算法设计中可以直接使用它们。

数据结构	说 明	实现头文件
向量（vector）	连续存储元素。底层数据结构为数组，支持快速随机访问	<vector>
字符串（string）	字符串处理容器	<string>
双端队列（deque）	连续存储的指向不同元素的指针所组成的数组。底层数据结构为一个中央控制器和多个缓冲区，支持首尾元素（中间不能）快速增删，也支持随机访问	<deque>
链表（list）	由结点组成的链表，每个结点包含着一个元素。底层数据结构为双向链表，支持结点的快速增删	<list>

数据结构	说 明	实现头文件
栈（stack）	后进先出的序列。底层一般用deque（默认）或者list实现	<stack>
队列（queue）	先进先出的序列。底层一般用deque（默认）或者list实现	<queue>
优先队列（priority_queue）	元素的进出队顺序由某个谓词或者关系函数决定的一种队列。底层数据结构一般为vector（默认）或者deque	<queue>
集合（set）/多重集合（multiset）	由结点组成的红黑树，每个结点都包含着一个元素，set中所有元素有序但不重复，multiset中所有关键字有序但可以重复	<set>
映射（map）/多重映射（multimap）	由（关键字，值）对组成的集合，底层数据结构为红黑树，map中所有关键字有序但不重复，multimap中所有关键字有序但可以重复	<map>

为此，使用STL时必须将下面的语句插入到源代码文件开头：

```
using namespace std;
```

这样直接把程序代码定位到std命名空间中。

2. 什么是STL算法

STL算法是用来操作容器中数据的模板函数，STL提供了大约100个实现算法的模版函数。例如，STL用sort()来对一个vector中的数据进行排序，用find()来搜索一个list中的对象。

STL算法部分主要由头文件<algorithm>、<numeric>和<functional>组成。

例如，以下程序使用STL算法sort()实现整型数组*a*的递增排序：

```
#include <algorithm>
using namespace std;
void main()
{  int a[]={2,5,4,1,3};
   sort(a,a+5);
   for (int i=0;i<5;i++)
            printf("%d ",a[i]);  //输出: 1 2 3 4 5
   printf("\n");
}
```

3. 什么是STL迭代器

STL迭代器用于访问容器中的数据对象。

每个容器都有自己的迭代器，只有容器自己才知道如何访问自己的元素。

迭代器像C/C++中的指针，算法通过迭代器来定位和操作容器中的元素。

常用的迭代器有：

- iterator：指向容器中存放元素的迭代器，用于正向遍历容器中的元素。
- const_iterator：指向容器中存放元素的常量迭代器，只能读取容器中的元素。
- reverse_iterator：指向容器中存放元素的反向迭代器，用于反向遍历容器中的元素。
- const_reverse_iterator：指向容器中存放元素的常量反向迭代器，只能读取容器中的元素。

迭代器的常用运算如下：

++：正向移动迭代器。

--：反向移动迭代器。

*：返回迭代器所指的元素值。

```
vector<int> myv;
myv.push_back(1);
myv.push_back(2);
myv.push_back(3);
vector<int>::iterator it;      //定义正向迭代器it
for (it=myv.begin();it!=myv.end();++it)
                                        //从头到尾遍历所有元素
        printf("%d ",*it);    //输出：1 2 3
printf("\n");
vector<int>::reverse_iterator rit;
                                        //定义反向迭代器rit
for (rit=myv.rbegin();rit!=myv.rend();++rit)
                                        //从尾到头遍历所有元素
        printf("%d ",*rit);  //输出：3 2 1
printf("\n");
```

1.3.2 常用的STL容器

- 顺序容器
- 适配器容器
- 关联容器

1. 顺序容器

1）vector（向量容器）

它是一个向量类模板。向量容器相当于数组。

用于存储具有相同数据类型的一组元素，可以从末尾快速的插入与删除元素，快速地随机访问元素，但是在序列中间插入、删除元素较慢，因为需要移动插入或删除处后面的所有元素。

定义vector容器的几种方式如下：

```
vector<int> v1;                    //定义元素为int的向量v1
vector<int> v2(10);                        //指定向量v2的初始大小为10个int元素
vector<double> v3(10，1.23);          //指定v3的10个初始元素的初值为1.23
vector<int> v4(a，a+5);     //用数组a[0..4]共5个元素初始化v4
```

vector提供了一系列的成员函数，vector主要的成员函数如下：

- empty()：判断当前向量容器是否为空。
- size()：返回当前向量容器的中的实际元素个数。
- []：返回指定下标的元素。
- reserve(n)：为当前向量容器预分配n个元素的存储空间。
- capacity()：返回当前向量容器在重新进行内存分配以前所能容纳的元素个数。
- resize(n)：调整当前向量容器的大小，使其能容纳n个元素。
- push_back()：在当前向量容器尾部添加了一个元素。
- insert(pos，elem)：在pos位置插入元素elem，即将元素elem插入到迭代器pos指定元素之前。

- front()：获取当前向量容器的第一个元素。
- back()：获取当前向量容器的最后一个元素。
- erase()：删除当前向量容器中某个迭代器或者迭代器区间指定的元素。
- clear()：删除当前向量容器中所有元素。
- 迭代器函数：begin()、end()、rbegin()、rend()。

示例

```
#include <vector>
using namespace std;
void main()
{   vector<int> myv;                    //定义vector容器myv
    vector<int>::iterator it;  //定义myv的正向迭代器it

    myv.push_back(1);                   //在myv末尾添加元素1
    it=myv.begin();             //it迭代器指向开头元素1
    myv.insert(it,2);           //在it指向的元素之前插入元素2
    myv.push_back(3);                   //在myv末尾添加元素3
    myv.push_back(4);                   //在myv末尾添加元素4
    it=myv.end();               //it迭代器指向尾元素4的后面
    it--;    //it迭代器指向尾元素4
    myv.erase(it);              //删除元素4

    for (it=myv.begin();it!=myv.end();++it)
            printf("%d ",*it);
    printf("\n");
}
```

2）string（字符串容器）

string是一个保存字符序列的容器，所有元素为字符类型，类似vector<char>。

除了有字符串的一些常用操作以外，还有包含了所有的序列容器的操作。字符串的常用操作包括增加、删除、修改、查找比较、连接、输入、输出等。

创建string容器的几种方式如下：

```
char cstr[]="China! Greate Wall";     //C-字符串
string s1(cstr);                      // s1:China! Greate Wall
string s2(s1);                                  // s2:China! Greate Wall
string s3(cstr，7，11);               // s3:Greate Wall
string s4(cstr，6);                             // s4:China!
string s5(5，'A');                    // s5:AAAAA
```

常用的成员函数如下：

- empty()：判断当前字符串是否为空串。
- size()：返回当前字符串的实际字符个数（返回结果为size_type类型）。
- length()：返回当前字符串的实际字符个数。
- [idx]：返回当前字符串位于idx位置的字符，idx从0开始。
- at(idx)：返回当前字符串位于idx位置的字符。
- compare(const string& str)：返回当前字符串与字符串str的比较结果。在比较时，若两者相等，返回0；前者小于后者，返回-1；否则返回1。
- append(cstr)：在当前字符串的末尾添加一个字符串str。
- insert(size_type idx，const string& str) ：在当前字符串的idx处插入一个字符串str。

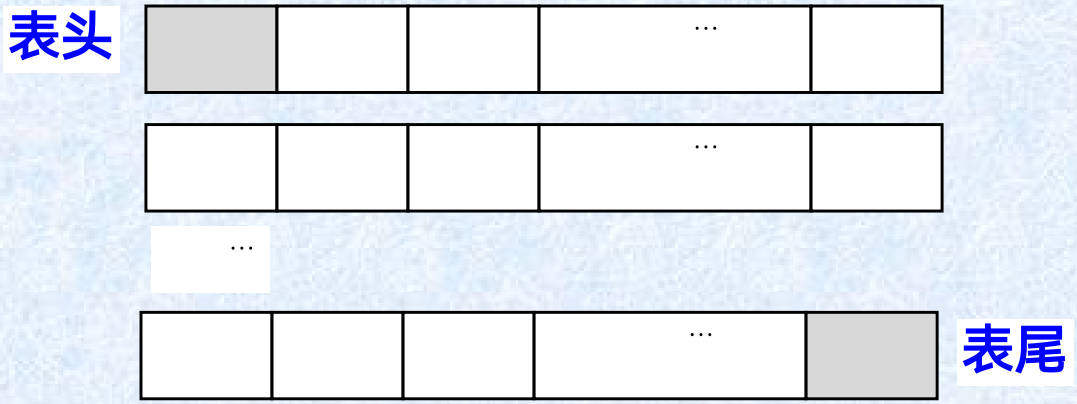
- empty()：判断当前字符串是否为空串。
- size()：返回当前字符串的实际字符个数（返回结果为size_type类型）。
- length()：返回当前字符串的实际字符个数。
- [idx]：返回当前字符串位于idx位置的字符，idx从0开始。
- at(idx)：返回当前字符串位于idx位置的字符。
- compare(const string& str)：返回当前字符串与字符串str的比较结果。在比较时，若两者相等，返回0；前者小于后者，返回-1；否则返回1。
- append(cstr)：在当前字符串的末尾添加一个字符串str。
- insert(size_type idx，const string& str)：在当前字符串的idx处插入一个字符串str。
- 迭代器函数：begin()、end()、rbegin()、rend()。

示例

```
#include <iostream>
#include <string>
using namespace std;
void main()
{   string s1="",s2,s3="Bye";
    s1.append("Good morning");      //s1=" Good morning"
    s2=s1;                          //s1=" Good morning"
    int i=s2.find("morning");       //i=5
    s2.replace(i,s2.length()-i,s3); //相当于s2.replace(5,7,s3)
    cout << "s1: " << s1 << endl;
    cout << "s2: " << s2 << endl;
}
```

3）deque（双端队列容器）

它是一个双端队列类模板。双端队列容器由若干个块构成，每个块中元素地址是连续的，块之间的地址是不连续的，有一个特定的机制将这些块构成一个整体。可以从前面或后面快速插入与删除元素，并可以快速地随机访问元素，但删除元素较慢。



定义deque双端队列容器的几种方式如下：

```
deque<int> dq1;  //定义元素为int的双端队列dq1
deque<int> dq2(10);          //指定dq2的初始大小为10个int元素
deque<double> dq3(10，1.23);
                             //指定dq3的10个初始元素的初值为1.23
deque<int> dq4(dq2.begin()，dq2.end());
                             //用dq2的所有元素初始化dq4
```

deque主要的成员函数如下：

- empty()：判断双端队列容器是否为空队。
- size()：返回双端队列容器中元素个数。
- push_front(elem)：在队头插入元素elem。
- push_back(elem)：在队尾插入元素elem。
- pop_front()：删除队头一个元素。
- pop_back()：删除队尾一个元素。
- erase()：从双端队列容器中删除一个或几个元素。
- clear()：删除双端队列容器中所有元素。
- 迭代器函数：begin()、end()、rbegin()、rend()。

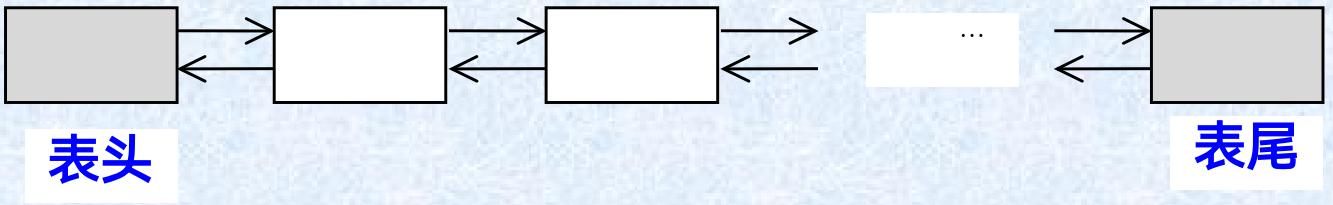
示例

```
#include <deque>
using namespace std;
void disp(deque<int> &dq)                    //输出dq的所有元素
{  deque<int>::iterator iter;                //定义迭代器iter
   for (iter=dq.begin();iter!=dq.end();iter++)
            printf("%d ",*iter);
   printf("\n");
}

void main()
{  deque<int> dq;                          //建立一个双端队列dq
   dq.push_front(1);                       //队头插入1
   dq.push_back(2);                        //队尾插入2
   dq.push_front(3);                       //队头插入3
   dq.push_back(4);                        //队尾插入4
   printf("dq: "); disp(dq);
   dq.pop_front();                         //删除队头元素
   dq.pop_back();                          //删除队尾元素
   printf("dq: "); disp(dq);
}
```

4）list（链表容器）

它是一个双链表类模板。可以从任何地方快速插入与删除。它的每个结点之间通过指针链接，不能随机访问元素。



定义list容器的几种方式如下：

```
list<int> l1;                                    //定义元素为int的链表l1
list<int> l2 (10);                    //指定链表l2的初始大小为10个int元素
list<double> l3 (10，1.23); //指定l3的10个初始元素的初值为1.23
list<int> l4(a，a+5);                          //用数组a[0..4]共5个元素初始化l4
```

list的主要成员函数如下：

- empty()：判断链表容器是否为空。
- size()：返回链表容器中实际元素个数。
- push_back()：在链表尾部插入元素。
- pop_back()：删除链表容器的最后一个元素。
- remove ()：删除链表容器中所有指定值的元素。
- remove_if(cmp)：删除链表容器中满足条件的元素。
- erase()：从链表容器中删除一个或几个元素。
- unique()：删除链表容器中相邻的重复元素。

- clear()：删除链表容器中所有的元素。
- insert(pos，elem)：在pos位置插入元素elem，即将元素elem插入到迭代器pos指定元素之前。
- insert(pos，n，elem)：在pos位置插入n个元素elem。
- insert(pos，pos1，pos2)：在迭代器pos处插入[pos1，pos2)的元素。
- reverse()：反转链表。
- sort()：对链表容器中的元素排序。
- 迭代器函数：begin()、end()、rbegin()、rend()。

说明：STL提供的sort()排序算法主要用于支持随机访问的容器，而list容器不支持随机访问，为此，list容器提供了sort()函数用于元素排序。类似的还有unique()、reverse()、merge()等STL算法。

示例

```
#include <list>
using namespace std;
void disp(list<int> &lst)                    //输出lst的所有元素
{  list<int>::iterator it;
   for (it=lst.begin();it!=lst.end();it++)
            printf("%d ",*it);
   printf("\n");
}
void main()
{  list<int> lst;                            //定义list容器lst
   list<int>::iterator it,start,end;
   lst.push_back(5);                                  //添加5个整数5,2,4,1,3
   lst.push_back(2);  lst.push_back(4);
   lst.push_back(1);  lst.push_back(3);
   printf("初始lst: "); disp(lst);
   it=lst.begin();                           //it指向首元素5
   start=++lst.begin();                      //start指向第2个元素2
   end=--lst.end();                                   //end指向尾元素1
   lst.insert(it,start,end);
   printf("执行lst.insert(it,start,end)\n");
   printf("插入后lst: "); disp(lst);
}
```

2. 关联容器

1）set（集合容器）/ multiset（多重集容器）

set和multiset都是集合类模板，其元素值称为关键字。set中元素的关键字是唯一的，multiset中元素的关键字可以不唯一，而且默认情况下会对元素按关键字自动进行升序排列。

查找速度比较快，同时支持集合的交、差和并等一些集合上的运算，如果需要集合中的元素允许重复那么可以使用multiset。

set/multiset的成员函数如下：

- empty()：判断容器是否为空。
- size()：返回容器中实际元素个数。
- insert()：插入元素。
- erase()：从容器删除一个或几个元素。
- clear()：删除所有元素。
- count(k)：返回容器中关键字k出现的次数。
- find(k)：如果容器中存在关键字为k的元素，返回该元素的迭代器，否则返回end()值。
- upper_bound()：返回一个迭代器，指向关键字大于k的第一个元素。
- lower_bound()：返回一个迭代器，指向关键字不小于k的第一个元素。
- 迭代器函数：begin()、end()、rbegin()、rend()。

示例

```
#include <set>
using namespace std;
void main()
{  set<int> s;                                    //定义set容器s
   set<int>::iterator it;          //定义set容器迭代器it
   s.insert(1);
   s.insert(3);
   s.insert(2);
   s.insert(4);
   s.insert(2);
   printf(" s: ");
   for (it=s.begin();it!=s.end();it++)
            printf("%d ",*it);
   printf("\n");
```

s:1 2 3 4

```
    multiset<int> ms;           //定义multiset容器ms
    multiset<int>::iterator mit;
                                //定义multiset容器迭代器mit
    ms.insert(1);
    ms.insert(3);
    ms.insert(2);
    ms.insert(4);
    ms.insert(2);
    printf("ms: ");
    for (mit=ms.begin();mit!=ms.end();mit++)
            printf("%d ",*mit);
    printf("\n");
}
```

ms:1 2 2 3 4

2）map（映射容器）/ multimap（多重映射容器）

map和multimap都是映射类模板。映射是实现关键字与值关系的存储结构，可以使用一个关键字key来访问相应的数据值value。

在set/multiset中的key和value都是key类型，而key和value是一个pair类结构。

pair类结构的声明形如：

```
struct pair
{   T first;
    T second;
}
```

map/multimap利用pair的<运算符将所有元素即key-value对按key的升序排列，以红黑树的形式存储，可以根据key快速地找到与之对应的value（查找时间为$O(\log_2 n)$）。

map中不允许关键字重复出现，支持[]运算符；而multimap中允许关键字重复出现，但不支持[]运算符。

map/multimap的主要成员函数如下：

- empty()：判断容器是否为空。
- size()：返回容器中实际元素个数。
- map[key]：返回关键字为key的元素的引用，如果不存在这样的关键字，则以key作为关键字插入一个元素（不适合multimap）。
- insert(elem)：插入一个元素elem并返回该元素的位置。
- clear()：删除所有元素。
- find()：在容器中查找元素。
- count()：容器中指定关键字的元素个数（map中只有1或者0）。
- 迭代器函数：begin()、end()、rbegin()、rend()。

在map中修改元素非常简单，这是因为map容器已经对[]运算符进行了重载。例如：

```
map<char,int> mymap;
                        //定义map容器mymap，其元素类型为pair<char,int>
mymap['a'] = 1;   //或者mymap.insert(pair<char，int>('a',1) );
```

获得map中一个值的最简单方法如下：

```
int ans = mymap['a'];
```

只有当map中有这个关键字（'*a*'）时才会成功，否则会自动插入一个元素，值为初始化值。可以使用find() 方法来发现一个关键字是否存在。

示例

```
#include <map>
using namespace std;
void main()
{   map<char,int> mymap;  //定义map容器mymap
    mymap.insert(pair<char,int>('a',1));
                                        //插入方式1
    mymap.insert(map<char,int>::value_type('b',2));
                                        //插入方式2
    mymap['c']=3;
                                //插入方式3
    map<char,int>::iterator it;
    for(it=mymap.begin();it!=mymap.end();it++)
            printf("[%c,%d] ",it->first,it->second);
    printf("\n");
}
```

[a,1] [b,2] [c,3]

3. 适配器容器

1）stack（栈容器）

它是一个栈类模板，和数据结构中的栈一样，具有后进先出的特点。栈容器默认的底层容器是deque。也可以指定其他底层容器。

例如，以下语句指定myst栈的底层容器为vector：

```
stack<string,vector<string> > myst;
                //第2个参数指定底层容器为vector
```

stack容器主要的成员函数如下：

- empty()：判断栈容器是否为空。
- size()：返回栈容器中实际元素个数。
- push(elem)：元素elem进栈。
- top()：返回栈顶元素。
- pop()：元素出栈。

注意： stack容器没有begin()/end()和rbegin()/rend()这样的用于迭代器的成员函数。

示例

```
#include <stack>
using namespace std;
void main()
{  stack<int> st;
   st.push(1); st.push(2); st.push(3);
   printf("栈顶元素: %d\n",st.top());
   printf("出栈顺序: ");
   while (!st.empty())          //栈不空时出栈所有元素
   {        printf("%d ",st.top());
            st.pop() ;
   }
   printf("\n");
}
```

2）queue（队列容器）

它是一个队列类模板，和数据结构中的队列一样，具有先进先出的特点。不允许顺序遍历，没有begin()/end()和rbegin()/rend()这样的用于迭代器的成员函数。

主要的成员函数如下：

- empty()：判断队列容器是否为空。
- size()：返回队列容器中实际元素个数。
- front()：返回队头元素。
- back()：返回队尾元素。
- push(elem)：元素elem进队。
- pop()：元素出队。

示例

```
#include <queue>
using namespace std;
void main()
{  queue<int> qu;
   qu.push(1); qu.push(2); qu.push(3);
   printf("队头元素: %d\n",qu.front());
   printf("队尾元素: %d\n",qu.back());
   printf("出队顺序: ");
   while (!qu.empty())                    //出队所有元素
   {        printf("%d ",qu.front());
            qu.pop();
   }
   printf("\n");
}
```

3）priority_queue（优先队列容器）

它是一个优先队列类模板。优先队列是一种具有受限访问操作的存储结构，元素可以以任意顺序进入优先队列。

一旦元素在优先队列容器中，出队操作将出队列最高优先级元素。

主要的成员函数如下：

- empty()：判断优先队列容器是否为空。
- size()：返回优先队列容器中实际元素个数。
- push(elem)：元素elem进队。
- top()：获取队头元素。
- pop()：元素出队。

示例

```
#include <queue>
using namespace std;
void main()
{   priority_queue<int> qu;
    qu.push(3); qu.push(1); qu.push(2);
    printf("队头元素: %d\n",qu.top());
    printf("出队顺序: ");
    while (!qu.empty())                    //出队所有元素
    {       printf("%d ",qu.top());
            qu.pop();
    }
    printf("\n");
}
```